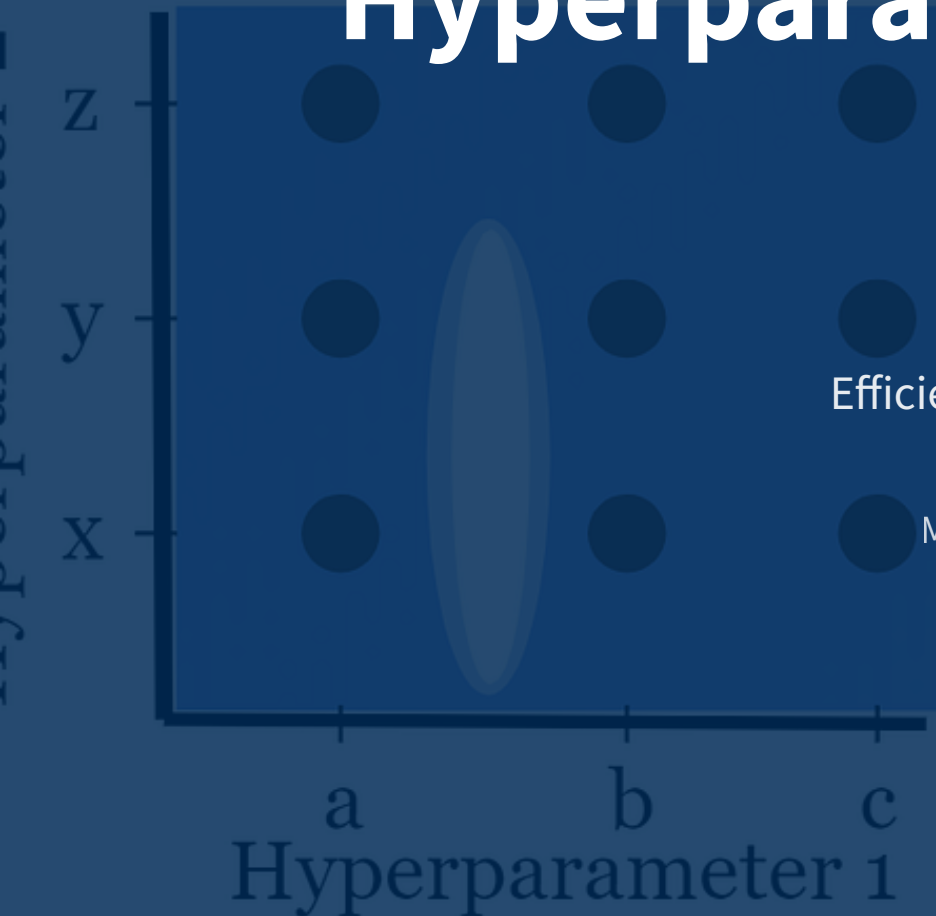


Grid Search

Pseudocode

```
Hyperparameter_One = [a, b, c]  
Hyperparameter_Two = [x, y, z]
```



Random Search

Pseudocode

```
Hyperparameter_One = random.num(r  
Hyperparameter_Two = random.num(r
```



Hyperparameter Tuning: Random Search

Efficiently Finding Optimal Model Parameters

Machine Learning Optimization Techniques

Introduction to Hyperparameters

Hyperparameters are configuration settings that are not learned during training but are set before the training process begins.

Parameters

Learned from data during training

VS

Hyperparameters

Set before training begins

Common Examples

- | | |
|--|---|
|  Learning Rate |  Number of Trees |
|  Regularization Strength |  Batch Size |
|  Max Depth |  Number of Hidden Layers |

Why Hyperparameters Matter

-  Directly impact model performance
-  Control bias-variance tradeoff
-  Affect training speed and efficiency
-  Determine model generalization ability
-  Require domain knowledge to set effectively

Hyperparameter Optimization

Search Space

The set of all possible hyperparameter combinations. Each dimension represents a different hyperparameter, and each point represents one model configuration.

Objective Function

A function that maps hyperparameters to model performance score. Goal: find hyperparameters that maximize/minimize this function.

$$\lambda^* = \operatorname{argmin}_{\lambda \in \Lambda} E_{x \sim G_x} [L(x, A_\lambda(X^{(\text{train})}))]$$

Find hyperparameters λ that minimize expected loss L

Key Challenges

- ▶ **High dimensionality** - exponential growth in search space
- ▶ **Computational cost** - evaluating each combination requires training a model
- ▶ **Noisy evaluations** - model performance varies with random initialization
- ▶ **Complex interactions** - hyperparameters often affect each other
- ▶ **Non-convex space** - many local optima, difficult to navigate

Curse of Dimensionality

With just 5 hyperparameters and 10 values each:

$$10^5 = 100,000$$

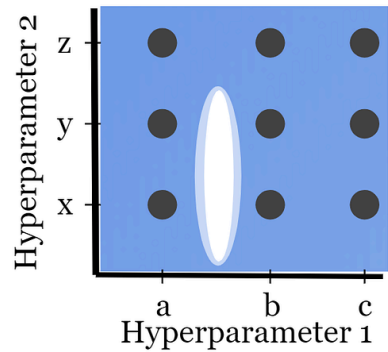
Possible combinations to evaluate!

Random Search vs. Grid Search

Grid Search

Pseudocode

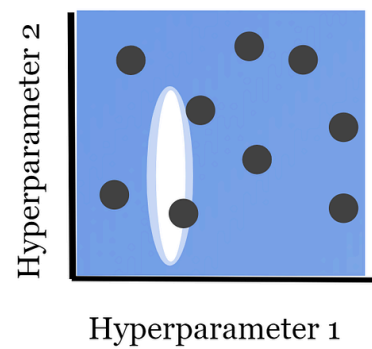
```
Hyperparameter_One = [a, b, c]  
Hyperparameter_Two = [x, y, z]
```



Random Search

Pseudocode

```
Hyperparameter_One = random.num(range)  
Hyperparameter_Two = random.num(range)
```



Random search explores more diverse combinations in the same time

Grid Search

Systematic, exhaustive

VS

Random Search

Random sampling

Grid Search

- ▶ Explores **all combinations** in a predefined grid
- ▶ Number of trials = product of all parameter values
- ▶ Suffers from **curse of dimensionality**
- ▶ Efficient in low-dimensional spaces

Random Search

- ▶ Randomly samples from **parameter distributions**
- ▶ Number of trials is independent of dimensions
- ▶ More efficient in **high-dimensional spaces**
- ▶ Unimportant parameters don't waste resources

How Random Search Works

<> Random Search Algorithm

- 1 Define Search Space**
Specify **ranges** or **distributions** for each hyperparameter
- 2 Sample Randomly**
Draw **n_iter** random combinations from the search space
- 3 Evaluate Each Combination**
Train model with each set using **cross-validation**
- 4 Select Best Parameters**
Choose combination with **highest performance** on validation set

Key Advantage

Budget is **independent** of number of parameters

🔧 Process Flow



Define hyperparameter distributions
e.g., `learning_rate ~ Uniform(0.001, 0.1)`



Randomly sample parameter combinations
n_iter = 100 iterations



Train model with each combination
5-fold cross-validation



Evaluate performance metrics
accuracy, F1-score, RMSE



Select best performing hyperparameters
Retrain on full training set

Theoretical Advantages of Random Search

📖 Bergstra & Bengio (2012)

"Random Search for Hyper-Parameter Optimization"

Seminal paper showing that **random search** is more efficient than **grid search** for hyperparameter optimization, especially in high-dimensional spaces.

🔧 Key Findings

- ✓ Random search finds models as good or better than grid search with **smaller computational budget**
- ✓ With equal budget, random search finds **better models** by exploring more diverse configurations
- ✓ Performance is often dominated by a **few important hyperparameters**

👤 Low Effective Dimensionality

Most performance functions have **low effective dimensionality** - they are sensitive to changes in only a few dimensions.

$$d < D$$

Where **d** = effective dimensions and **D** = total dimensions

$$f(x_1, x_2, \dots, x_D) \approx g(x_1, x_2, \dots, x_d)$$

Random search explores **all dimensions** while grid search wastes resources on unimportant ones.

"Random search is a natural baseline against which to judge progress in the development of adaptive hyperparameter optimization algorithms."

— Bergstra & Bengio (2012)

Implementation in scikit-learn

<> RandomizedSearchCV Usage

```
from sklearn.model_selection import
RandomizedSearchCV

# Define model model =
RandomForestClassifier()

# Define search space param_dist = {
'n_estimators': [100, 200, 300, 400],
'max_depth': [5, 10, 15, 20, None],
'min_samples_split': [2, 5, 10],
'max_features': ['sqrt', 'log2'] }

# Create random search random_search =
RandomizedSearchCV( estimator=model,
param_distributions=param_dist, n_iter=100,
cv=5, scoring='accuracy', n_jobs=-1 )

# Fit and find best parameters
random_search.fit(X_train, y_train)
```

Key Advantage

Budget is **independent** of number of parameters

⚙️ Key Parameters

estimator

Model to optimize

param_distributions

Dictionary of parameter ranges

n_iter

Number of parameter settings
sampled

cv

Cross-validation folds

scoring

Performance metric

n_jobs

Parallel jobs (-1 = all CPUs)

≡ Defining Parameter Space

Two approaches to specify parameters:

```
# Discrete values 'max_features': ['sqrt',
'log2'] # Distributions from scipy.stats
import uniform 'C': uniform(0.1, 10)
```

Use distributions for continuous parameters to explore
more values with increasing n_iter

Practical Example

<> Random Search with RandomForest

```
# Import libraries from sklearn.ensemble
import RandomForestClassifier from
sklearn.model_selection import
RandomizedSearchCV from
sklearn.model_selection import
train_test_split import numpy as np # Load
and split data X_train, X_test, y_train,
y_test = train_test_split(X, y,
test_size=0.2) # Define parameter space
param_dist = { 'n_estimators': [int(x) for
x in np.linspace(200, 2000, 10)],
'max_features': ['auto', 'sqrt'],
'max_depth': [int(x) for x in
np.linspace(10, 110, 11)],
'min_samples_split': [2, 5, 10],
'min_samples_leaf': [1, 2, 4], 'bootstrap':
[True, False] } # Create random search
object random_search = RandomizedSearchCV(
estimator=RandomForestClassifier(),
param_distributions=param_dist, n_iter=100,
cv=5, verbose=2, scoring="accuracy",
random_state=42, n_jobs=-1 ) # Fit the
model random_search.fit(X_train, y_train)
```

Implementation Steps

- 1 **Import** necessary libraries and load data
- 2 **Split** data into training and testing sets
- 3 **Define** parameter space with ranges
- 4 **Create** RandomizedSearchCV object
- 5 **Fit** the model to training data
- 6 **Extract** best parameters and results

Extracting Results

After fitting, access the best parameters and performance:

```
# Get best parameters best_params =
random_search.best_params_ # Get best
estimator best_model =
random_search.best_estimator_ # Evaluate on
test set test_accuracy =
best_model.score(X_test, y_test)
print(f"Best Parameters: {best_params}")
print(f"Test Accuracy:
{test_accuracy:.4f}")
```

Best Practices for Random Search

Key Recommendations

- ✓ Use **distributions** for continuous parameters
- ✓ Set **n_iter** based on computational budget
- ✓ Choose appropriate **cross-validation** strategy
- ✓ Consider **parallel processing** with n_jobs
- ✓ Start with **wide ranges**, then narrow down

Remember

Random search finds **good enough** parameters efficiently, not necessarily the absolute best

Practical Tips

Parameter Distributions

Use **log-uniform** for scale-sensitive parameters (learning rate, regularization)

Budget Allocation

n_iter = 60-100 typically balances exploration and computation time

Cross-Validation

5-fold CV for most cases, use **stratified** for imbalanced datasets

Performance

Set **n_jobs=-1** to use all CPU cores for faster execution

Comparison Results

✂ When to Use Random Search

- ✓ **High-dimensional** parameter spaces
- ✓ Limited **computational budget**
- ✓ When only **few parameters matter**
- ✓ For **initial exploration** of parameter space

📊 When to Use Grid Search

- ✓ **Low-dimensional** parameter spaces
- ✓ When you need to **systematically explore** all combinations
- ✓ When parameters have **known interactions**
- ✓ For **fine-tuning** around promising regions

📊 Performance Comparison

5-10×

Faster exploration in high dimensions

80%

Parameters often have low impact

60-100

Optimal n_iter for most cases

3-5×

Better results with same budget

🧠 Decision Framework

Consider both the **dimensionality** and **computational budget**

Random Search

Efficient exploration of diverse configurations

VS

Grid Search

Systematic evaluation of all combinations

For most real-world scenarios with multiple parameters, **start with random search**

Advanced Topics

Successive Halving

Combines random search with resource allocation strategy:

SH

Allocates **more resources** to promising configurations while discarding poor performers early.

How Successive Halving Works

- 1 Sample many configurations with **minimal resources**
- 2 Evaluate and **rank** configurations by performance
- 3 Keep only **top performers** (e.g., $1/\eta$ of candidates)
- 4 Allocate **more resources** to remaining candidates
- 5 Repeat until resources are exhausted

Advanced Optimization Techniques

Bayesian Optimization

Uses probabilistic model to guide search toward promising regions

Evolutionary Algorithms

Applies principles of natural selection to evolve better solutions

Gradient-Based Optimization

Uses gradient information to navigate parameter space

Multi-Fidelity Methods

Combines cheap approximations with expensive evaluations

Implementation in scikit-learn

HalvingRandomSearchCV combines random search with successive halving for more efficient hyperparameter optimization

Conclusion

Key Points

- ✓ Random search **outperforms** grid search in high-dimensional spaces
- ✓ More **efficient** due to low effective dimensionality
- ✓ Computational budget is **independent** of number of parameters
- ✓ Easy to implement with **RandomizedSearchCV** in scikit-learn

Recommendation

Use **random search** as your baseline for hyperparameter optimization

n_iter = 60-100

For most practical scenarios

Key Takeaways



Faster exploration in high dimensions



Better resource allocation



Use distributions for continuous parameters



Good baseline for advanced methods

Final Thought

"Random search is a natural baseline against which to judge progress in the development of adaptive hyperparameter optimization algorithms."

— *Bergstra & Bengio (2012)*